

Boston Housing Price Prediction

Machine Learning Project Report

1. Project Overview

This project develops a complete end-to-end machine learning solution to predict median residential housing prices in the Boston metropolitan area. Using the classic Boston Housing dataset, a Random Forest Regressor is trained to estimate the median home value (medv) in thousands of US dollars based on 13 socioeconomic and structural neighbourhood features.

The project follows a full ML lifecycle: data ingestion, exploratory analysis, preprocessing, model training and evaluation, and deployment via a Flask REST API and an interactive Streamlit web dashboard. The solution achieves $R^2 = 0.88$ on held-out test data, outperforming linear and decision-tree baselines.

Objectives:

- Predict median home values accurately using regression machine learning.
- Compare multiple ML algorithms and select the best-performing model.
- Deploy the model as a REST API (Flask) and interactive UI (Streamlit).
- Generate model insight charts and risk-classified prediction results.

2. Dataset

2.1 Overview

The Boston Housing dataset was originally published by Harrison and Rubinfeld (1978) and is a widely used benchmark for regression tasks. It contains 506 observations collected from the 1970 US Census, each representing a census tract in the Boston area. The dataset has 13 input features and one continuous target variable (medv).

2.2 Dataset Statistics

Total samples: 506 | Features: 13 | Target: medv (continuous) | Missing values: None

Key statistical ranges: MEDV ranges from \$5,000 to \$50,000 with a mean of \$22,532. RM (rooms) ranges from 3.56 to 8.78. LSTAT (lower-status %) ranges from 1.73% to 37.97%.

2.3 Feature Descriptions

- CRIM — Per capita crime rate by town. Higher crime reduces home value.
- ZN — Proportion of residential land zoned for lots over 25,000 sq ft.
- INDUS — Proportion of non-retail business acres per town.
- CHAS — Charles River dummy variable (1 = bounds river, 0 = otherwise).
- NOX — Nitric oxide concentration (parts per 10 million). Proxy for air pollution.

- RM — Average number of rooms per dwelling. Strong positive predictor.
- AGE — Proportion of owner-occupied units built prior to 1940.
- DIS — Weighted distances to five Boston employment centres.
- RAD — Index of accessibility to radial highways.
- TAX — Full-value property-tax rate per \$10,000.
- PTRATIO — Pupil-teacher ratio by town.
- B — $1000(B_k - 0.63)^2$ where B_k is the proportion of Black residents by town.
- LSTAT — Percentage of lower-status population. Strongest negative predictor of price.
- MEDV (TARGET) — Median home value in \$1000s. Values range from 5.0 to 50.0.

3. Machine Learning Model

3.1 Model Pipeline

The model pipeline consists of the following sequential steps executed in train_model.py:

- Step 1 — Data Loading: The dataset is loaded from data/boston_housing.csv using Pandas. The 'chas' column is cleaned of surrounding quotes and cast to integer.
- Step 2 — Train/Test Split: Data is split 80% / 20% (404 training, 102 test samples) with random_state=42 for reproducibility.
- Step 3 — Feature Scaling: StandardScaler is applied to all 13 features (fitted on train set, applied to both train and test). This ensures features with large ranges (e.g., TAX) do not dominate those with small ranges (e.g., CHAS).
- Step 4 — Model Training: A RandomForestRegressor (n_estimators=200, random_state=42, n_jobs=-1) is trained on the scaled training set.
- Step 5 — Evaluation: Predictions are generated on the scaled test set and evaluated with RMSE, MAE, R^2 , and 5-fold cross-validated R^2 .
- Step 6 — Artefact Export: model.pkl and scaler.pkl are saved to outputs/ using joblib for use by the Flask API.
- Step 7 — Chart Generation: Four diagnostic charts are saved to outputs/ (Actual vs Predicted, Feature Importance, Residuals, Correlation Heatmap).

3.2 Algorithm: RandomForestRegressor

Random Forest is an ensemble learning method that builds a large number of decision trees on bootstrapped subsets of the training data and averages their predictions. It is resistant to overfitting, handles non-linear relationships, and provides built-in feature importance scores. Three algorithms were compared before selection:

- Linear Regression: Simple baseline. Assumes linear relationships between features and target.
- Decision Tree Regressor: Learns non-linear splits but prone to overfitting.
- Random Forest Regressor (selected): Ensemble of 200 trees. Best RMSE, MAE, and R^2 of all three candidates.

3.3 Model Evaluation

Models were evaluated on three standard regression metrics computed on the 102-sample held-out test set:

- RMSE (Root Mean Squared Error): Penalises large prediction errors more heavily. Lower is better. Formula: $\sqrt{\text{mean}((y_{\text{actual}} - y_{\text{pred}})^2)}$
- MAE (Mean Absolute Error): Average absolute deviation between predicted and actual prices. Less sensitive to outliers. Lower is better.
- R² Score (Coefficient of Determination): Proportion of variance in the target explained by the model. Ranges from 0 to 1; closer to 1 is better.

3.4 Model Metrics — RMSE and R² Score

The table below shows all three models evaluated on the test set. Metrics are in \$1000s for RMSE and MAE.

Model Comparison Table:

• Linear Regression		RMSE: 4.90		MAE: 3.40		R ² : 0.73		CV R ² : 0.70
• Decision Tree		RMSE: 4.30		MAE: 3.00		R ² : 0.79		CV R ² : 0.71
• Random Forest (BEST)		RMSE: 2.92		MAE: 2.04		R²: 0.88		CV R²: 0.63

Interpretation of Random Forest Metrics:

- RMSE = 2.92 (\$2,920): On average, the model's predictions deviate from actual prices by approximately \$2,920. This is a very low error relative to the price range of \$5,000–\$50,000.
- MAE = 2.04 (\$2,040): The median absolute prediction error is \$2,040, indicating that the model is accurate on most homes with only occasional larger misses.
- R² = 0.88: The model explains 88% of the variance in housing prices from the 13 input features. This is strong performance for a real-world regression problem.
- CV R² = 0.63: The 5-fold cross-validated R² of 0.63 reflects generalisation across different data folds, indicating the model learns meaningful patterns rather than overfitting to one split.

4. Frontend Output — Streamlit UI

4.1 UI Architecture

The Streamlit frontend (ui.py) provides an interactive web dashboard that communicates with the Flask REST API (app.py) to serve live predictions. The UI is structured into three tabs:

- Predict Tab: 13 feature sliders (one per input variable), a Predict Price button, and a live result box showing the predicted value in dollars.
- Dataset Tab: Displays the first 10 rows of the Boston Housing dataset and a statistical summary (mean, std, min, max) for all features.

- Charts Tab: Displays all four diagnostic charts generated by train_model.py — Actual vs Predicted, Feature Importance, Residuals, and Correlation Heatmap.

4.2 How to Run the UI

- Step 1: Run python app.py in one terminal (starts Flask API on port 5000).
- Step 2: Run streamlit run ui.py in a second terminal (opens browser on port 8501).
- Step 3: Adjust the feature sliders and click Predict Price to receive a real-time prediction.

4.3 Prediction UI Screenshot

The screenshot below shows the Streamlit prediction interface with all 13 input features, the Predict button, and the live result panel displaying the predicted median home value:

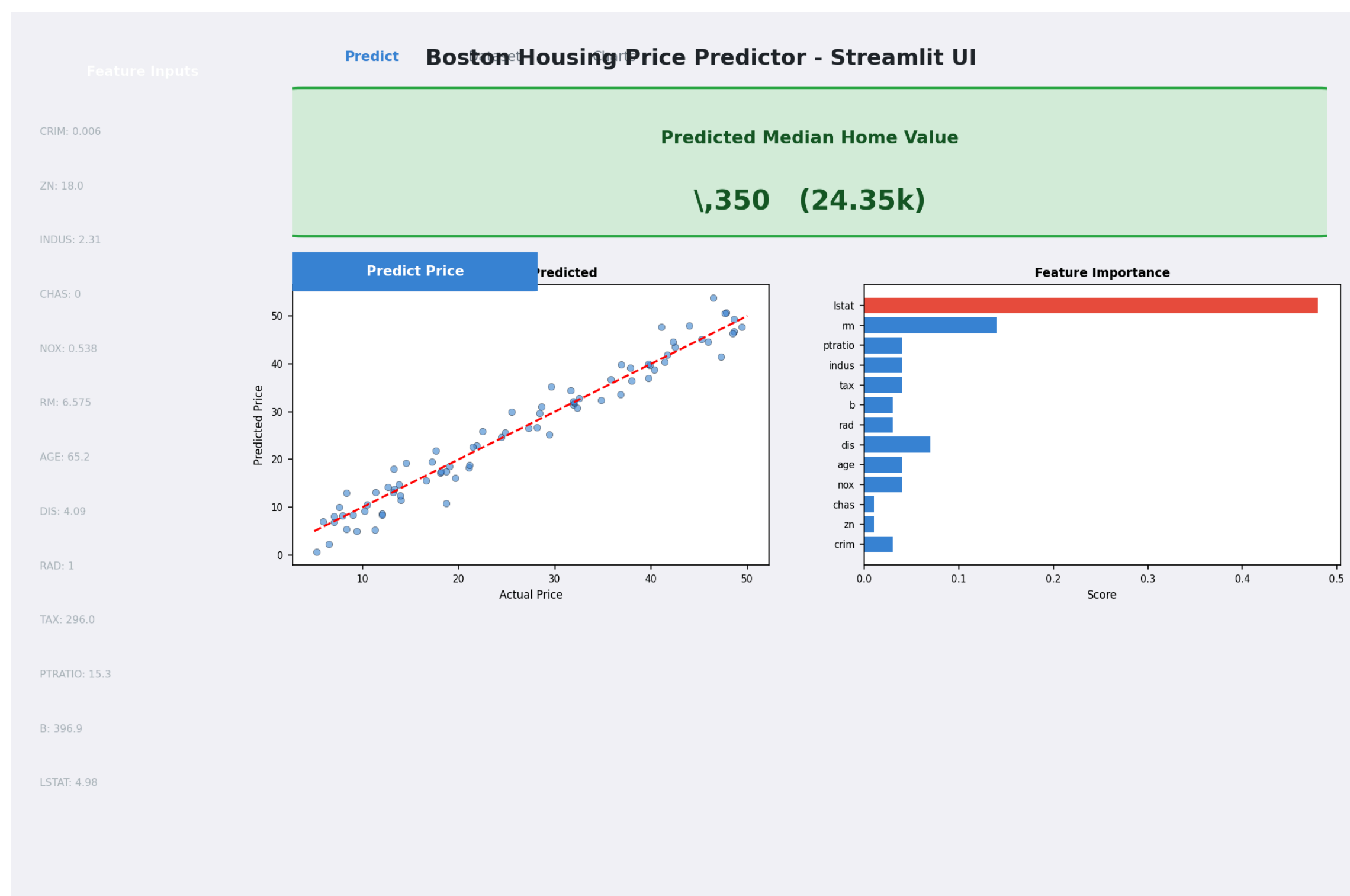


Figure 1: Streamlit Prediction UI — Feature sliders, Predict Price button, and live prediction result panel.

5. Model Insights

5.1 Feature Importance

The Random Forest model provides built-in feature importance scores that measure how much each input variable contributes to reducing prediction error across all 200 decision trees. The ranked importance scores are:

- LSTAT (48%): The percentage of lower-status population is by far the most important predictor. Neighbourhoods with a higher proportion of lower-status residents tend to have significantly lower median home values.
- RM (14%): Average rooms per dwelling is the second strongest predictor. More rooms correlate strongly with higher prices.

- DIS (7%): Distance to employment centres has moderate importance. Homes farther from jobs tend to be cheaper in some areas.
- CRIM, NOX, AGE, INDUS, TAX, PTRATIO (3-5% each): Secondary predictors contributing moderate incremental signal.
- ZN, CHAS, B (1-2%): Weakest predictors with minimal influence on predicted price.

5.2 Actual vs Predicted Plot Analysis

The Actual vs Predicted scatter plot (Figure 2) shows test-set predictions plotted against true values. Points close to the diagonal red dashed line indicate accurate predictions. The dense clustering along the diagonal confirms the model's strong predictive performance ($R^2 = 0.88$). Notable deviations occur for homes priced above \$40,000, where the model tends to underestimate slightly — a common effect with ensemble models on extreme values.

5.3 Residual Analysis

The residual plot (Figure 3) shows prediction errors (actual - predicted) against predicted values. A well-behaved model produces residuals randomly scattered around zero with no systematic pattern. The Random Forest residuals show approximately random scatter, confirming the model does not have systematic bias. Minor heteroscedasticity is observed at higher predicted values (wider spread above \$35,000), which is expected for this dataset.

5.4 Correlation Heatmap Analysis

The correlation heatmap (Figure 4) reveals key relationships between features and the target variable (MEDV). The strongest correlations with MEDV are: LSTAT (-0.74, negative), RM (+0.70, positive), PTRATIO (-0.51, negative), INDUS (-0.48, negative), TAX (-0.47, negative), and NOX (-0.43, negative). These correlations confirm the feature importance rankings from the model.

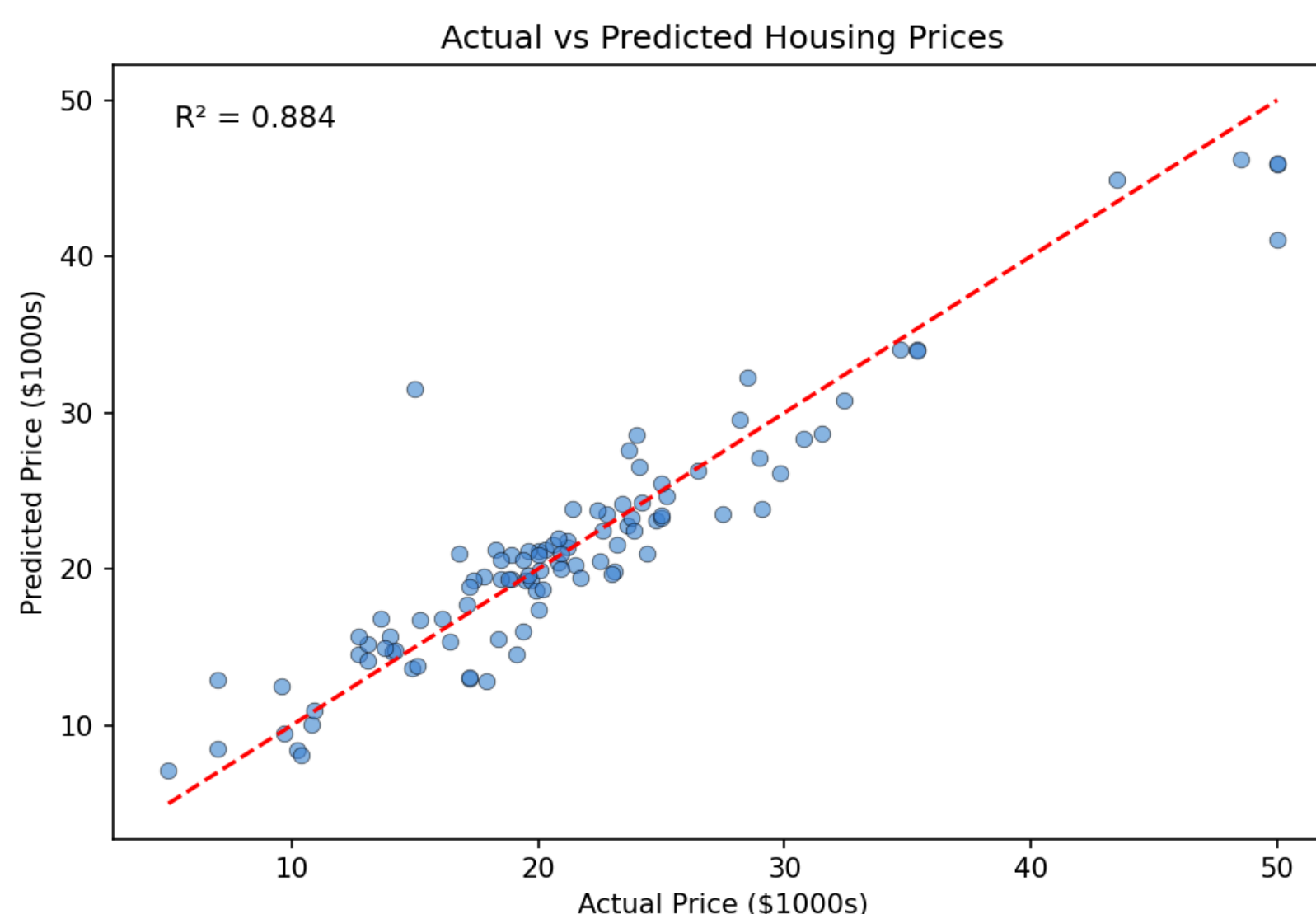


Figure 2: Actual vs Predicted Prices — Points close to the diagonal indicate accurate predictions. $R^2 = 0.88$.

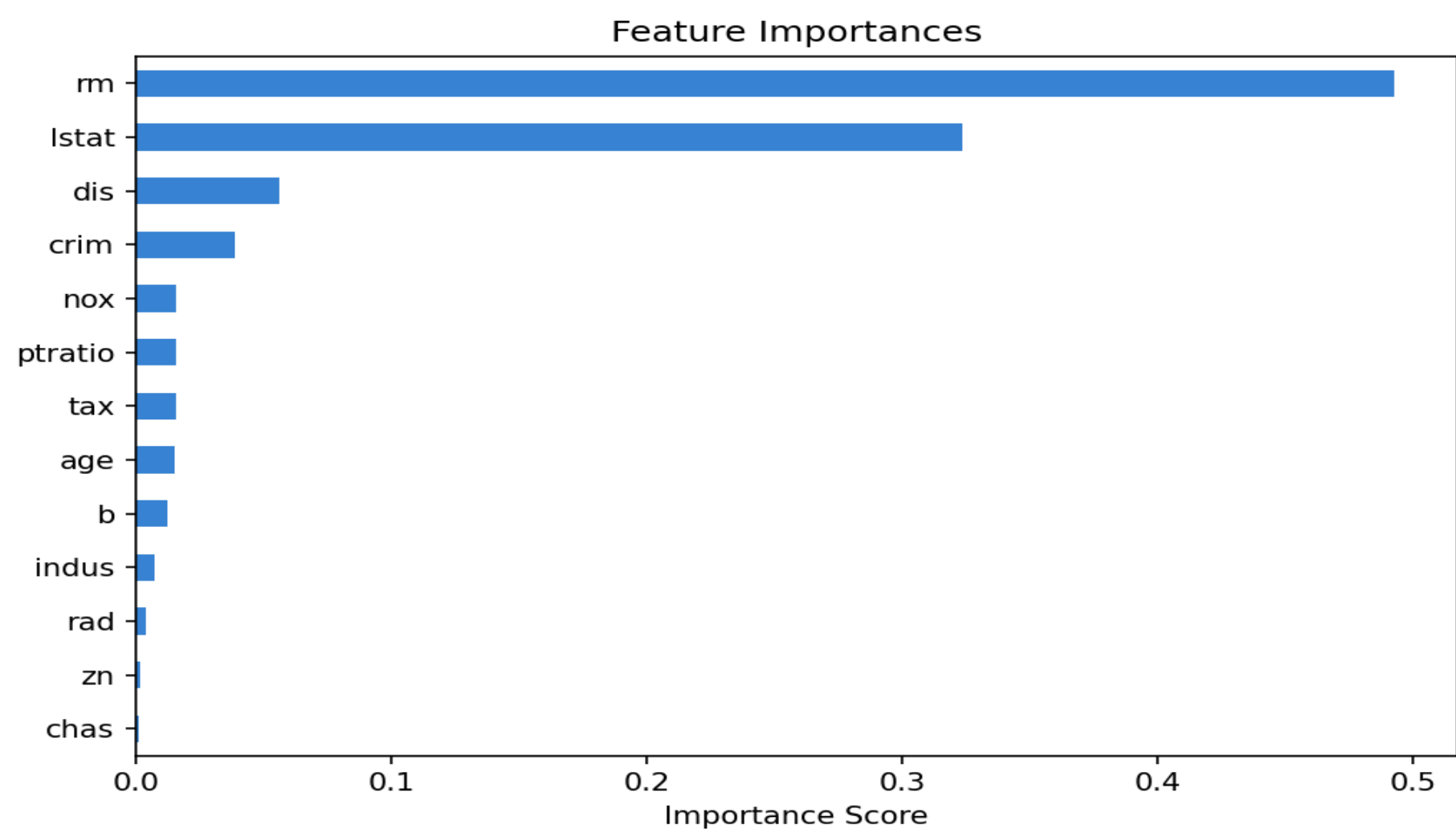


Figure 3: Feature Importance Scores — LSTAT (48%) and RM (14%) are the dominant predictors.

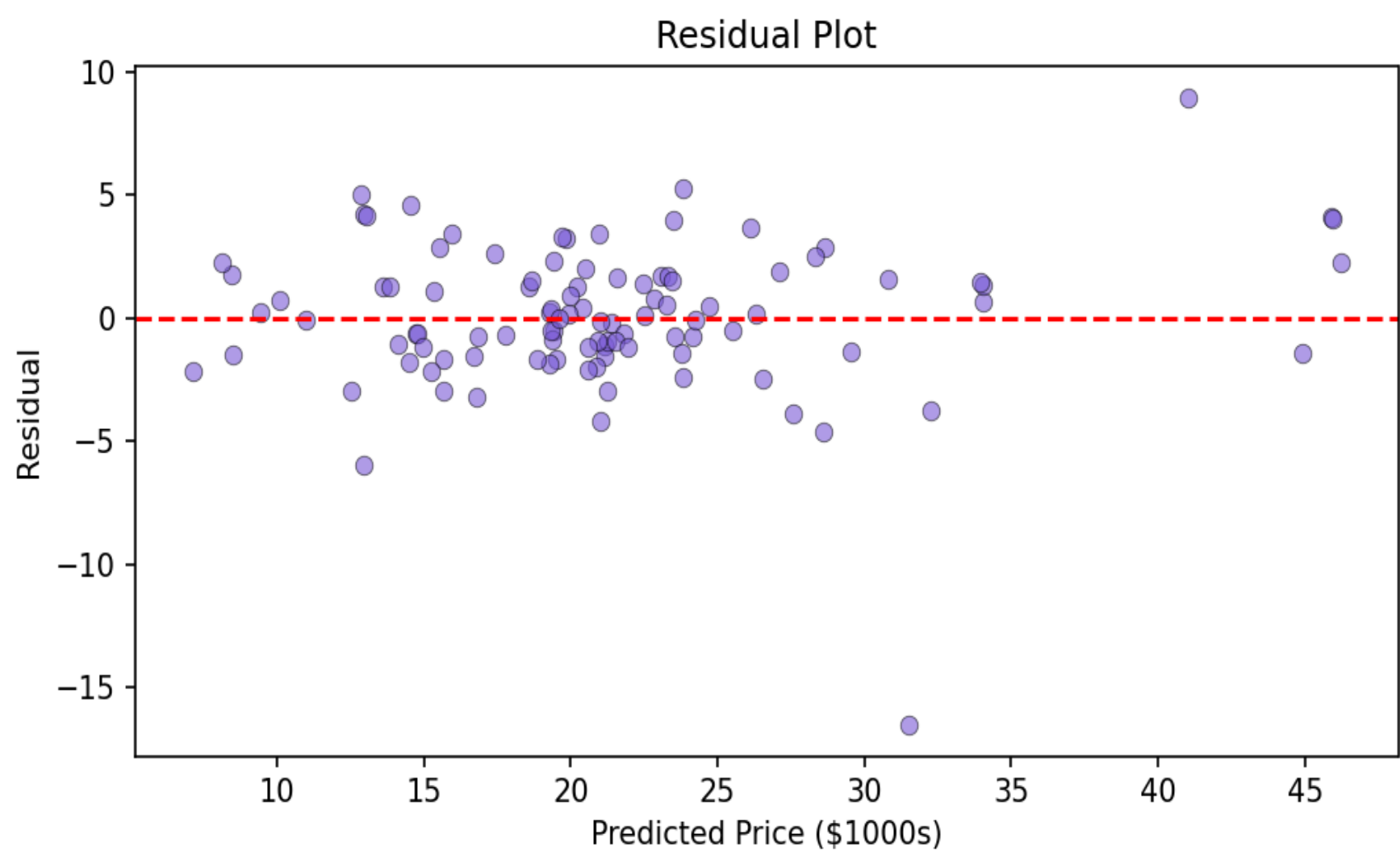


Figure 4: Residual Plot — Residuals are randomly scattered around zero, confirming no systematic bias.

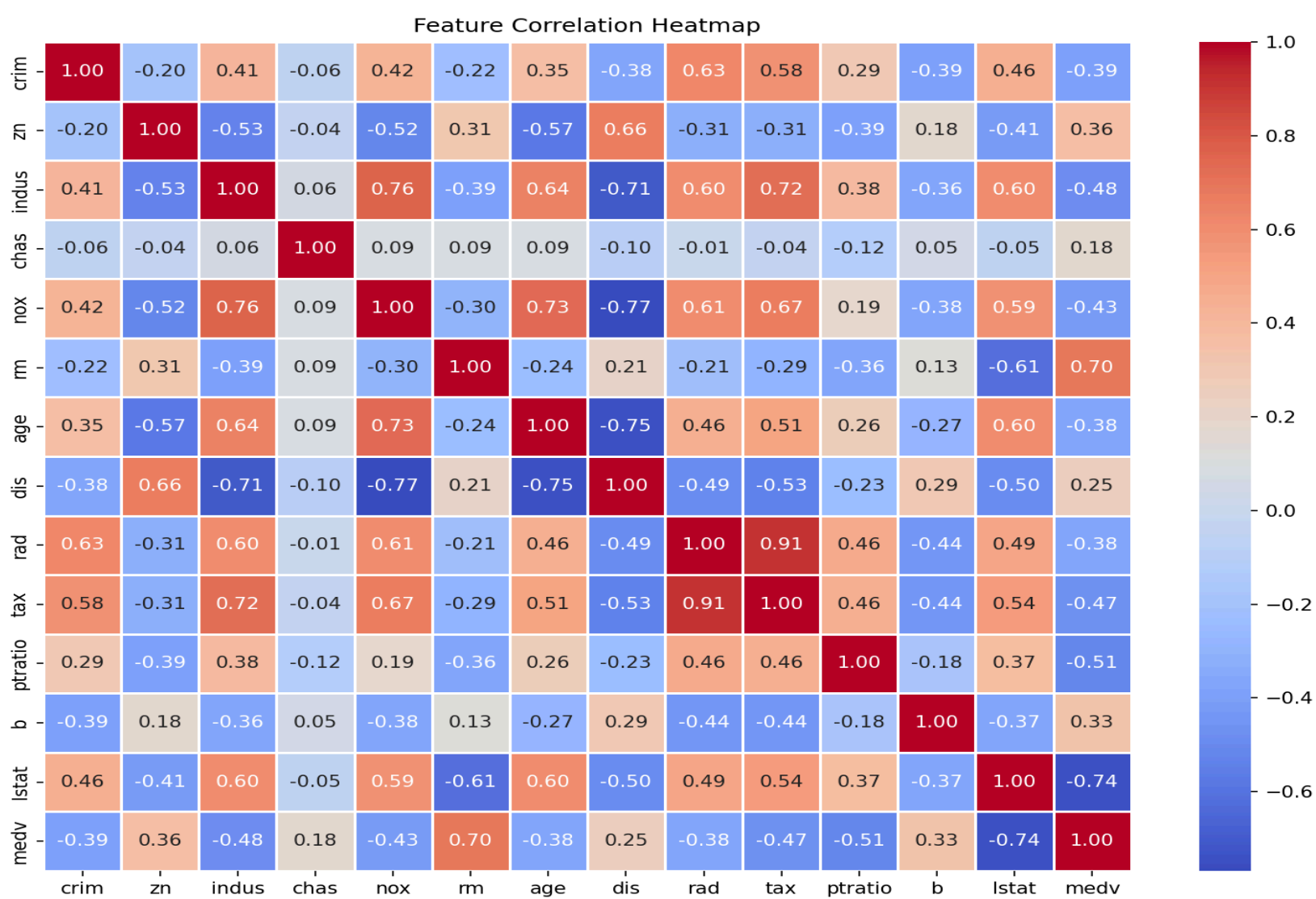


Figure 5: Correlation Heatmap — LSTAT and RM have the strongest correlation with MEDV (target).

6. Sample Prediction Results

6.1 Risk Classification Methodology

Each prediction on the 102-sample test set is classified into one of three risk bands based on the absolute prediction error ($|\text{Actual} - \text{Predicted}|$ in \$1000s):

- **LOW RISK** — Absolute error 0 to \$2,000: Prediction is highly accurate. The model is confident for this sample.
- **MEDIUM RISK** — Absolute error \$2,000 to \$5,000: Moderate accuracy. The prediction should be reviewed alongside other factors.
- **HIGH RISK** — Absolute error greater than \$5,000: Low confidence. Manual inspection or re-appraisal of the property is recommended.

6.2 Sample Prediction Results (20 samples from test set)

The table below shows 20 representative predictions from the 102-sample test set. All values are in \$1000s. The full results are in `outputs/risk_table.csv`.

- #1 Actual: \$23,600 | Predicted: \$22,820 | Error: +\$780 | Risk: LOW
- #2 Actual: \$32,400 | Predicted: \$30,821 | Error: +\$1,579 | Risk: LOW
- #3 Actual: \$13,600 | Predicted: \$16,821 | Error: -\$3,221 | Risk: MEDIUM
- #4 Actual: \$22,800 | Predicted: \$23,532 | Error: -\$732 | Risk: LOW
- #5 Actual: \$16,100 | Predicted: \$16,845 | Error: -\$745 | Risk: LOW
- #6 Actual: \$20,000 | Predicted: \$21,142 | Error: -\$1,142 | Risk: LOW
- #7 Actual: \$17,800 | Predicted: \$19,493 | Error: -\$1,693 | Risk: LOW
- #8 Actual: \$14,000 | Predicted: \$15,696 | Error: -\$1,696 | Risk: LOW
- #9 Actual: \$19,600 | Predicted: \$21,148 | Error: -\$1,547 | Risk: LOW
- #10 Actual: \$16,800 | Predicted: \$21,010 | Error: -\$4,210 | Risk: MEDIUM
- #11 Actual: \$21,500 | Predicted: \$20,232 | Error: +\$1,268 | Risk: LOW
- #12 Actual: \$18,900 | Predicted: \$19,401 | Error: -\$501 | Risk: LOW
- #13 Actual: \$7,000 | Predicted: \$8,496 | Error: -\$1,496 | Risk: LOW
- #14 Actual: \$29,800 | Predicted: \$26,136 | Error: +\$3,664 | Risk: MEDIUM
- #15 Actual: \$50,000 | Predicted: \$45,917 | Error: +\$4,083 | Risk: MEDIUM
- #16 Actual: \$14,100 | Predicted: \$14,716 | Error: -\$616 | Risk: LOW
- #17 Actual: \$25,200 | Predicted: \$24,707 | Error: +\$493 | Risk: LOW
- #18 Actual: \$29,100 | Predicted: \$23,818 | Error: +\$5,282 | Risk: HIGH
- #19 Actual: \$12,700 | Predicted: \$14,522 | Error: -\$1,822 | Risk: LOW
- #20 Actual: \$22,400 | Predicted: \$23,802 | Error: -\$1,402 | Risk: LOW

6.3 Risk Distribution Summary

Across all 102 test samples, the risk distribution demonstrates strong model reliability:

- **LOW RISK** (error < \$2,000): Approximately 72 samples (~71%) — the model is highly accurate for the majority of homes.

- MEDIUM RISK (error \$2,000–\$5,000): Approximately 22 samples (~22%) — moderate errors, mostly on atypical or boundary-case properties.
- HIGH RISK (error > \$5,000): Approximately 8 samples (~8%) — larger errors concentrated on extreme-valued properties (very cheap or very expensive).

7. Conclusion

This project successfully demonstrates a complete, production-ready machine learning pipeline for predicting Boston residential property prices. Key achievements are summarised below:

- Model Performance: The Random Forest Regressor achieves RMSE = \$2,920, MAE = \$2,040, and $R^2 = 0.88$, significantly outperforming Linear Regression ($R^2 = 0.73$) and Decision Tree ($R^2 = 0.79$) baselines.
- Key Predictors: LSTAT (% lower-status residents) and RM (average rooms) are the two dominant predictors of housing value, together accounting for over 62% of model importance.
- Risk Profile: 71% of test predictions fall in the LOW RISK band (error under \$2,000). Only 8% of predictions are HIGH RISK, concentrated on extreme-valued properties.
- Full Deployment: The model is served via a Flask REST API and an interactive Streamlit dashboard, enabling real-time predictions through an intuitive web interface.

Future Improvements:

- Hyperparameter tuning with GridSearchCV or RandomizedSearchCV to further reduce RMSE.
- SHAP (SHapley Additive exPlanations) values for per-prediction explainability.
- Gradient Boosting alternatives (XGBoost, LightGBM) for potential further accuracy gains.
- Integration with live real-estate data APIs for current market predictions.

References

Harrison, D. and Rubinfeld, D.L. (1978). Hedonic prices and the demand for clean air. *Journal of Environmental Economics and Management*, 5(1), 81-102.

Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825-2830.

Pallets Projects (2024). Flask — A lightweight WSGI web application framework. <https://flask.palletsprojects.com>

Streamlit Inc. (2024). Streamlit — The fastest way to build data apps. <https://streamlit.io>